CSS

Dumitrita Sorohan

Abstract

1 Personal Contribution

My contribution to the project focused on the implementation and validation of the process execution subsystem of the operating system scheduler simulator. More specifically, I worked on the development of the `ExecutionStage` and `Process` classes, together with the corresponding unit tests and runtime assertions required in the later project phases.

The implemented components are responsible for simulating the behaviour of user and system processes, handling execution stages, system calls, CPU affinity tracking, and process memory state management.

2 Phase 1 – Application Development

2.1 ExecutionStage Class

The `ExecutionStage` class was designed to represent one execution segment of a user process. Each stage contains:

- the number of CPU execution ticks;
- the duration of the following system call;
- or `None` if the stage is the final execution segment.

For example, the sequence:

```
(5 2 3 4 6)
```

is interpreted as:

- execute for 5 ticks, then perform a system call of 2 ticks;
- execute for 3 ticks, then perform a system call of 4 ticks;
- execute for 6 ticks and terminate.

The class also performs validation of the received values in order to prevent invalid execution stages from being created.

2.2 Process Class

The `Process` class represents both user processes and the special system process responsible for executing system calls.

The implementation supports:

- parsing execution sequences into execution stages;
- execution progress through CPU ticks;

- transitions between execution stages;
- system call management;
- CPU affinity tracking;
- memory residency simulation.

2.2.1 Execution Sequence Parsing

One important part of the implementation was transforming the flat execution sequence into structured execution stages.

For example:

```
[5, 2, 3, 4, 6]
```

is internally converted into:

Stage	CPU Burst	System Call
1	5	2
2	3	4
3	6	None

This functionality was implemented inside the private method:

```
_parse_sequence()
```

2.2.2 Process Execution Logic

The execution logic was implemented through methods such as:

```
tick()
advance()
is_done()
is_waiting_for_sys_call()
```

The `tick()` method advances the execution of the process by one unit of time, while `advance()` moves the process to the next execution stage after a system call finishes.

The implementation distinguishes clearly between user processes and the system process, since their runtime behaviour is different.

2.2.3 System Call Queue

For the system process, I implemented a queue-based mechanism for handling system calls requested by user processes.

System calls are stored using the helper class:

```
SysCallRequest
```

The queue allows the system process to execute requests sequentially and resume the blocked user processes after completion.

2.2.4 CPU Affinity Tracking

Another implemented feature was CPU affinity tracking. Since the scheduler specification states that a process should preferably continue execution on the processor it used most recently, each process stores a history of CPU assignments.

The following methods were implemented:

```
record_cpu()
get_last_cpu()
get_cpu_history()
```

2.2.5 Memory State Management

The process implementation also contains support for memory residency simulation.

The methods:

```
load_into_memory()
evict_from_memory()
is_in_memory()
```

allow the scheduler and memory manager to determine whether a process is currently loaded into RAM or stored on disk.

3 Phase 2 – Unit Testing

For testing, I used Python's built-in `unittest` framework.

The goal of the tests was to verify:

- correct parsing of execution sequences;
- correct execution behaviour;
- proper stage transitions;
- correct system call handling;
- CPU history functionality;
- memory state transitions;
- handling of invalid inputs and edge cases.

3.1 ExecutionStage Tests

The tests for `ExecutionStage` verify:

- valid object creation;
- invalid execution durations;
- invalid system call durations;
- correct handling of final stages;
- string representation behaviour.

Example:

```
with self.assertRaises(ValueError):
    ExecutionStage(0, 2)
```

3.2 Process Tests

The `Process` class tests cover both user and system process behaviour.

The tested scenarios include:

- execution sequence parsing;
- decrementing execution ticks;
- process completion;
- waiting for system calls;

- queueing multiple system calls;
- triggering user process advancement after system call completion;
- CPU history recording;
- memory loading and eviction.

Several edge cases were also tested, such as invalid stage advancement or executing a system process with an empty syscall queue.

4 Phase 3 – Assertions

Assertions were added directly into the implementation code in order to validate program correctness during runtime.

The assertions were mainly used for:

- preconditions;
- postconditions;
- class invariants;
- state consistency checks.

4.1 Assertions in ExecutionStage

The constructor validates that execution durations are always positive.

Example:

```
assert run_ticks > 0
assert sys_call_ticks is None or sys_call_ticks > 0
```

4.2 Assertions in Process

A dedicated method named:

```
_check_invariants()
```

was implemented to verify the internal consistency of a process object.

The invariants ensure that:

- process identifiers are valid;
- execution stages remain consistent;
- runtime values never become negative;
- stage indexes remain inside valid bounds.

Example invariant:

```
assert 0 <= self.left_to_run <= current.run_ticks
```

Additional assertions were inserted into methods such as:

```
tick()
advance()
add_sys_call()
record_cpu()
```

These checks helped identify invalid runtime states during development and debugging.

5 Conclusion

Through this contribution, I implemented the core process execution functionality required by the operating system scheduling simulator.

The developed subsystem provides:

- process execution modelling;
- system call handling;
- execution stage management;
- CPU affinity tracking;
- memory state simulation;
- runtime correctness validation using assertions;
- extensive unit testing coverage.

The implementation was intentionally kept modular and relatively simple in order to facilitate integration with the scheduler, memory manager, and graphical interface developed in the other parts of the project.